UNIFIELD / ENGINEERING DESIGN

System Design Document

Vehicle Document Intelligence Portal

Version 1.0 | 28 October 2026

One vehicle. One clear report.

The portal brings sales documents and service history into one place. A user enters a vehicle registration number, reviews the records, and downloads one PDF. They can also correct report details or add notes before saving a copy.

This document explains the design of the current project and the next steps needed for a wider rollout. It is written for people who need to understand the system without reading its code.

What the current version includes

- A simple search using an Indian registration number, such as MH12AB1234.
- Two API calls made together: one for sales and documents, and one for service records.
- A combined PDF, a service timeline, report edits, and PDF download.

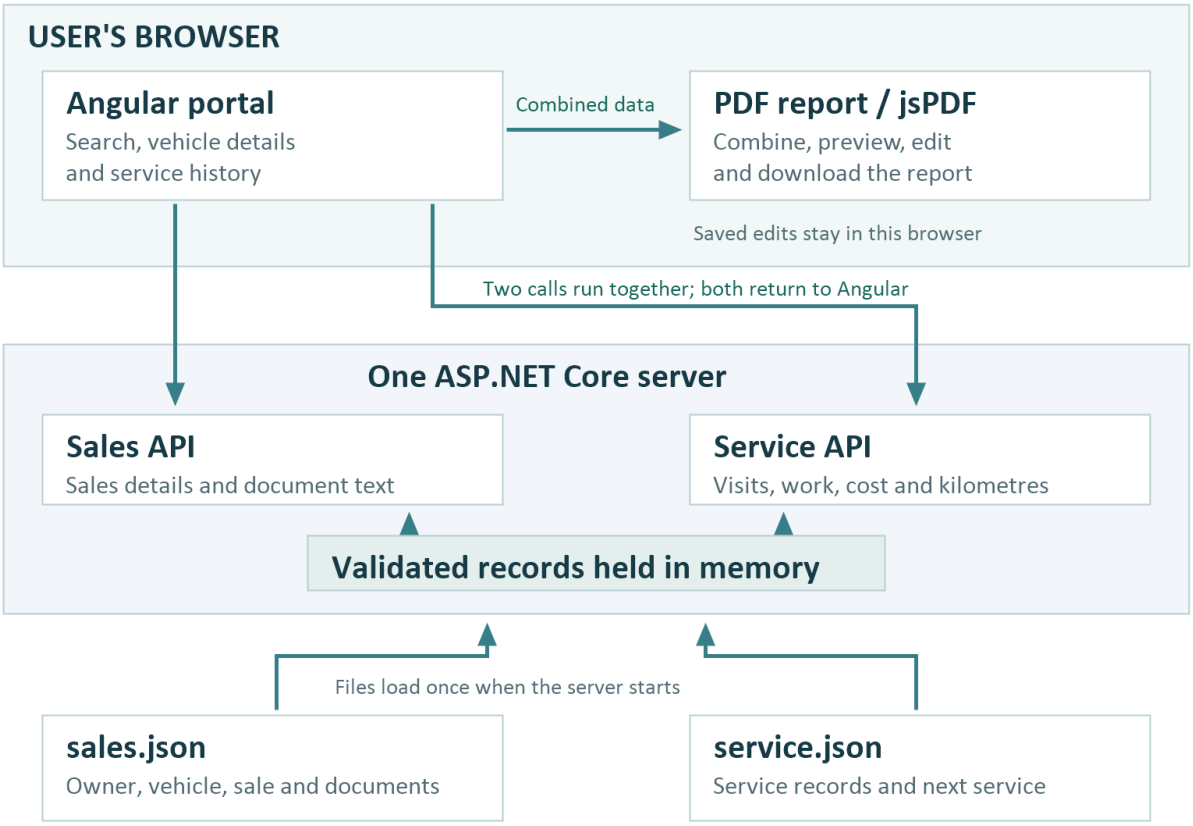
Design at a glance

One .NET server serves the Angular website and the APIs. Two JSON files hold 50 vehicle records across 19 brands. Report edits can be saved into json again for future use.

01 / ARCHITECTURE

How the system fits together

The design keeps the first version small and easy to run. Sales and service are separate API routes inside the same application. They are not separate deployed services.



Current design: one server, two logical APIs, two separate data files.

Figure 1. Current architecture. API calls run together; the browser builds the final report.

Why this shape works

The website and APIs share one server address. This keeps local setup simple and avoids a separate cross-origin configuration. Keeping sales and service data apart makes it easier to connect each API to its own source later.

02 / COMPONENTS

What each part does

Component	Main role
Angular portal	Collects the registration number and shows vehicle details, sales documents, the service timeline, and report controls.
ASP.NET Core host	Serves the built website and API routes. Checks requests, handles errors, and adds request IDs and logs.
Sales API	Finds the vehicle sale, owner, dealership, price, invoice, ownership record, and warranty text.
Service API	Finds dated visits, odometer readings, work performed, costs, technicians, and the next service.
JSON data files	Keep sales and service records separate. The server loads and checks both files when it starts.
PDF builder	Uses jsPDF in the browser to create a paginated report from both API responses and any manual notes.
Browser storage	Keeps report edits for each registration number on that browser. It does not save changes on the server.

Where the data lives

The project avoids Realtime DB connections and hence The server holds a read-only in-memory copy of the JSON mock data. Changing a JSON file takes effect after the server restarts. The browser receives only the records requested by the user, apart from the small vehicle list used for selection.

The PDF is created in browser memory and shown in the browser's PDF viewer. Downloading it saves a file to the user's device. Old preview links are released when a report is replaced or the page closes.

03 / DATA FLOW

From a number plate to a PDF

1. Enter and check the registration

The user enters MH12AB1234, MH 12 AB 1234, or a lowercase version. The portal removes spaces and hyphens, changes letters to uppercase, and checks the format.

2. Start both requests together

Angular calls the sales and service APIs at the same time. Each request has a 15-second timeout. The server returns an X-Request-ID header so a failed call can be found in the logs.

3. Look up the two records

Each API checks the normalized registration and searches its own in-memory records. A bad format returns 400. A valid number with no record returns 404. An unexpected server failure returns a safe 500 response.

4. Combine and display the result

When both calls succeed, Angular checks that the records match the requested vehicle. It loads saved browser edits, creates the PDF, shows the service timeline, and refreshes the screen immediately.

5. Edit the report if needed

The user can correct report details or add notes. The portal creates a replacement PDF and saves the edits in browser storage. If PDF creation fails, the previous report remains available.

6. Download a copy

The user downloads the PDF. If browser storage is unavailable, the report can still be downloaded. Browser edits do not carry across devices.

API routes

GET route	Returns
/api/vehicles	Registration, make, model, and year for the selection list.
/api/sales/{registrationNumber}	Sales details and document text.
/api/services/{registrationNumber}	Service history and next-service details.
/api/health	Application status and the loaded vehicle count.

Both sources must succeed before a unified report is shown. The portal does not silently present a partial report as complete.

04 / TECHNOLOGY

Tools chosen for this project

Technology	Why it was chosen
ASP.NET Core 9 / C#	A small API host with built-in routing, logging, static-file hosting, and JSON support. It fits the installed Visual Studio version.
Angular 19.2 / TypeScript 5.8	Supports forms and clear UI structure. This version works with the existing Node.js 22.9 installation.
esbuild	Builds the Angular code into static files. This project uses a small custom build rather than the Angular CLI.
jsPDF 4.2.1	Creates the report in the browser, so PDF generation does not need another backend service.
JSON / System.Text.Json	Keeps the initial data easy to read, review, and regenerate. Separate files reflect the two data sources.
Browser localStorage	Keeps report edits available after refresh without adding a write API or database to the first version.
en-IN locale	Formats rupee values with Indian digit grouping. The UI uses the rupee symbol; PDFs use INR for font compatibility.
Node.js 22.9 / npm 10.8	Uses the tools already installed on the development machine. No global upgrade or downgrade is needed.

05 / OBSERVABILITY

How we find and understand problems

Observability means being able to tell what the system is doing and why something failed. The first version has useful logs and request IDs. Metrics dashboards and full tracing are planned additions.

Logging - in place

The backend writes structured JSON logs to the terminal or Visual Studio Output. Each entry has a UTC time, level, and event details. Request logs include the request ID, route template, status code, and elapsed time.

Key events cover data loading, validation, startup, shutdown, successful lookups, invalid inputs, missing records, cancellations, and unexpected errors. Broken or inconsistent source files stop startup with a critical log.

The browser console logs searches, API timing, PDF generation, download preparation, edit saves, and storage failures. Routine logs avoid registration numbers, owner names, document text, and notes. Server errors shown to users do not contain stack traces.

Metrics - proposed

Measure	What it would tell us
Request count and 5xx rate	Whether the APIs are available and returning unexpected errors.
95th-percentile response time	How slow the slower requests are for sales and service lookups.
Search and PDF failure counts	Whether users can complete the main task.
Health-check availability	Whether the portal is reachable and has started successfully.

Add OpenTelemetry metrics and send them to a chosen monitoring service. As an initial alert rule, investigate if 5xx errors exceed 5% for five minutes, with at least 20 requests. Review this threshold after real usage is known.

Tracing - partial today, fuller tracing proposed

Today, each API response has a request ID that can be matched to backend logs. The two parallel calls have separate IDs; there is no shared end-to-end search trace. Add a shared search ID and OpenTelemetry spans to follow a search through both calls and PDF creation. Do not use registrations as trace or metric labels.

Logs currently go to consoles only. Central storage, access controls, and retention rules need to be agreed before a wider rollout.

06 / RELIABILITY

Failure handling and next steps

What happens when something goes wrong

- Bad input: show a clear example and do not send a search request.
- No matching vehicle: show a not-found message and clear the loading state.
- Slow or failed API: stop after the timeout and allow the user to retry.
- Failed PDF edit: keep the previous PDF and explain that the update failed.
- Full or blocked browser storage: keep the generated PDF available to download.

Checks already used

Build checks cover the backend and frontend. API checks exercise all 50 registrations, invalid numbers, unknown numbers, normalized inputs, request IDs, and unknown API routes.

Other checks cover screen-refresh notifications, timeout and network handling, malformed JSON, PDF rollback, download failure, log privacy, invoice totals, service dates, and increasing odometer readings. Backend tests also check critical startup logs when a data file is invalid.

These checks do not replace a browser review of layout, accessibility, or the final PDF appearance. Those should be checked before release.

Plan before wider use

Add sign-in and role-based access before exposing personal vehicle records. Use HTTPS, protect any connected-system credentials, and control who can read operational logs. Local browser edits are convenient, but they are not an audit trail.

For shared editing, add a database and a versioned report-edit API. Record who changed what and when. Keep source records separate from report corrections. Replace JSON lookups with dealer and service-system connections when those sources are available.

Scale only when needed: a shared database comes before multiple server copies; separate services come later if sales and service need independent ownership or capacity.

07 / GENAI IN THE DESIGN PHASE

How GenAI helped shape the design

GenAI acted as a collaborative design assistant during this project, helping to streamline the development process and refine key architectural decisions based on user requirements. Rather than driving the project, the AI assisted in reducing UI complexity, establishing a simpler overall design, and vetting technical choices as the portal took shape

Turning the request into a plan

The project initiated with a requirement for a .NET and Angular setup involving two data sources, a unified PDF output, and editable report details. GenAI helped break these requirements down into distinct components: the browser UI, sales API, service API, data files, a PDF builder, and browser storage. To minimize initial setup complexity and keep the application lightweight, the AI suggested grouping both APIs into a single server for the first version, making it significantly easier to run and test.

Using feedback to improve the design

User feedback changed the search from VINs to Indian registration numbers and changed money and distance to INR and kilometres. GenAI also helped identify the missing screen-refresh notification, improve failure handling, and separate npm and Visual Studio build outputs.

The assistant helped create varied test records and suggest checks for invoice totals, service order, PDF errors, and logs. Generated records remain fictional; realistic wording does not make them verified vehicle records.

Project Boundaries

The AI functioned strictly as an advisory tool. It did not connect the portal to live vehicle databases, verify ownership records, or provide final production approval. The resulting application runs entirely on standard code without calling an AI model, executing OCR, or processing uploaded documents. All architectural and design choices require final validation by the project owner prior to production rollout, and standard security reviews and user testing remain essential.